

Instituto Tecnológico y de Estudios Superiores de Monterrey



Escuela de Ingeniería y Ciencias

Ingeniería en Ciencia de Datos y Matemáticas

Manual de Desarrollo

Gestor de Identidades y Documentos

Versión v1.0

Socio Formador: Casa Monarca, Ayuda Humanitaria al Migrante, A.B.P.

Profesores: Raúl Gómez, Anas Wajid y Alberto F. Martínez

Equipo de trabajo: Equipo 5

Grupo: 603

Integrantes

Alberto Rodríguez Reyes (A01383805)

Diego Alberto Rodríguez Ruiz (A01571638)

Rubén Jiménez Sarmiento (A01286256)

Hector Alonso Flores Melendez (A01571545)

Valeria García Hernández (A01742811)

Lugar y fecha de elaboración: Monterrey, Nuevo León, 15 de junio de 2026

Índice general

1. Descripción General	1
1.1. Propósito del sistema	1
1.2. Alcance	1
1.3. Casos de uso principales	2
2. Arquitectura del Sistema	3
2.1. Tipo de arquitectura	3
2.2. Diagrama general	3
2.3. Componentes principales	4
3. Stack Tecnológico	5
3.1. Lenguajes	5
3.2. Frameworks y herramientas principales	5
3.3. Bibliotecas y dependencias	6
4. Estructura del Proyecto	7
4.1. Organización de carpetas	7
4.2. Descripción de módulos	7
4.2.1. Módulo <code>api/</code> — Backend PHP	7
4.2.2. Módulo <code>src/pages/</code> — Páginas React	8
5. Configuración del Entorno	9
5.1. Requisitos de versiones	9
5.2. Variables de entorno	9
5.3. Archivos de configuración	10
6. Instalación y Ejecución	11
6.1. Pasos para levantar el sistema	11
6.2. Comandos principales	12
7. Base de Datos	13
7.1. Descripción general	13

7.2. Tablas principales	13
7.2.1. Tabla usuarios	13
7.2.2. Tabla migrantes	13
7.2.3. Tabla documentos	14
7.2.4. Tabla documentos_historial	14
7.2.5. Tabla documentos_firmas	14
7.2.6. Tabla solicitudes_push	15
7.3. Relaciones entre tablas	15
8. API / Interfaces	16
8.1. Convención general	16
8.2. Endpoints principales	16
8.2.1. POST /api/login.php	16
8.2.2. POST /api/avanzar_documento.php	17
8.2.3. POST /api/firmar_documento.php	17
8.2.4. POST /api/generar_llaves_coordinador.php	17
9. Flujos y Lógica de Negocio	18
9.1. Flujo documental VOCA	18
9.2. Reglas de negocio importantes	18
9.3. Proceso de firma digital RSA	19
10. Ejemplos de Uso	20
10.1. Autenticar un usuario (login)	20
10.2. Subir y avanzar un documento	21
10.3. Firmar un documento digitalmente	21
10.4. Generar llaves RSA para un coordinador	22
10.5. Verificar la integridad de un documento firmado	23
11. Manejo de Errores	24
11.1. Estrategia de errores	24
11.2. Mensajes de error comunes	24
11.3. Registro (logs)	25
12. Pruebas	26
12.1. Tipos de pruebas	26
12.2. Casos de prueba manuales realizados	26
12.3. Cómo ejecutar ESLint	26
13. Despliegue	28
13.1. Entornos	28

13.2. Pasos para despliegue en producción	28
13.3. Consideraciones y limitaciones conocidas (HostGator)	28
14. Seguridad	30
14.1. Autenticación	30
14.2. Firma digital	30
14.3. Protección de datos sensibles	30
14.4. Buenas prácticas implementadas	31
15. Guía de Contribución	32
15.1. Convenciones de código	32
15.1.1. Frontend (JavaScript/React)	32
15.1.2. Backend (PHP)	32
15.2. Flujo de trabajo con Git	32
15.3. Cómo agregar funcionalidades	33
16. Problemas Conocidos	34
16.1. Limitaciones de la versión 1.0	34
17. FAQ Técnica	35
18. Referencias	36

Índice de tablas

1.1. Casos de uso principales del sistema	2
2.1. Componentes principales del sistema	4
3.1. Lenguajes de programación utilizados	5
3.2. Frameworks y herramientas del proyecto	5
3.3. Dependencias del proyecto (package.json)	6
4.1. Endpoints del backend PHP	8
4.2. Páginas del frontend React	8
5.1. Requisitos de software por componente	9
5.2. Archivos de configuración del proyecto	10
6.1. Comandos npm disponibles	12
7.1. Relaciones entre tablas de la base de datos	15
8.1. Parámetros de login.php	16
8.2. Parámetros de avanzar_documento.php	17
8.3. Parámetros de firmar_documento.php	17
8.4. Parámetros de generar_llaves_coordinador.php	17
11.1. Errores comunes del sistema y causa raíz	24
12.1. Casos de prueba funcionales manuales	26
13.1. Entornos del sistema	28
16.1. Limitaciones conocidas del sistema	34

Índice de figuras

2.1. Diagrama de arquitectura del sistema	3
4.1. Estructura de carpetas del proyecto	7
9.1. Flujo documental VOCA	18

Capítulo 1

Descripción General

1.1. Propósito del sistema

El **Gestor de Identidades y Documentos** es una aplicación web desarrollada para **Casa Monarca**, organización civil de atención a personas migrantes en Monterrey, Nuevo León. El sistema centraliza la gestión de usuarios, migrantes, solicitudes y documentos internos, con énfasis en el flujo de revisión y firma digital de documentos oficiales mediante criptografía asimétrica (RSA).

El propósito central es reemplazar procesos manuales y en papel por un flujo digital controlado, trazable y seguro, que garantice que cada documento pase por las etapas de revisión y aprobación correctas antes de ser finalizado.

1.2. Alcance

El sistema abarca las siguientes funcionalidades principales:

- **Autenticación y control de acceso** por roles (admin, coordinador, operador, consulta).
- **Gestión de usuarios**: creación, edición, activación/desactivación y asignación de coordinador.
- **Registro de migrantes**: captura de datos personales, migratorios y de atención.
- **Gestor de documentos**: carga, flujo por etapas ($V \rightarrow O \rightarrow C \rightarrow A$), firma digital RSA y verificación de integridad.
- **Solicitudes push**: flujo de aprobación delegado entre usuarios y coordinadores.
- **Generación de llaves RSA** para coordinadores, con clave privada cifrada por contraseña.

El sistema **no incluye**: módulo de reportes estadísticos avanzados, integración con sistemas externos de migración ni notificaciones por correo electrónico.

1.3. Casos de uso principales

Tabla 1.1: Casos de uso principales del sistema

Caso de uso	Rol principal	Descripción
Iniciar sesión	Todos	Autenticación con email y contraseña. Validación de estado y vigencia.
Gestionar usuarios	Admin	Crear, editar, activar/desactivar usuarios y asignar coordinadores.
Registrar migrante	Admin, coordinador, operador	Capturar datos completos de un migrante atendido.
Subir documento	Consulta, otros	Registrar un documento en el sistema e iniciar el flujo documental.
Aprobar documento (V→O)	Operador, admin	Avanzar documento desde Ventanilla hacia revisión operativa.
Firmar documento (O→C)	Coordinador	Firmar digitalmente con clave privada RSA y contraseña.
Inserción final (C→A)	Admin	Cerrar el flujo documental una vez completadas todas las firmas.
Verificar firma	Todos	Comprobar la autenticidad e integridad de la firma digital.
Generar llaves RSA	Admin, coordinador	Generar par de llaves RSA para el coordinador, cifrado con contraseña.

Capítulo 2

Arquitectura del Sistema

2.1. Tipo de arquitectura

El sistema sigue una arquitectura de **cliente-servidor desacoplada**, donde el frontend y el backend son procesos independientes que se comunican mediante HTTP/JSON:

- **Frontend:** aplicación de página única (SPA) construida con React y servida por Vite.
- **Backend:** conjunto de endpoints PHP independientes, ejecutados por Apache (XAMPP).
- **Base de datos:** MySQL/MariaDB gestionada por XAMPP.
- **Proxy de desarrollo:** Vite redirige las llamadas a `/api/*` hacia Apache en el puerto 80.

2.2. Diagrama general

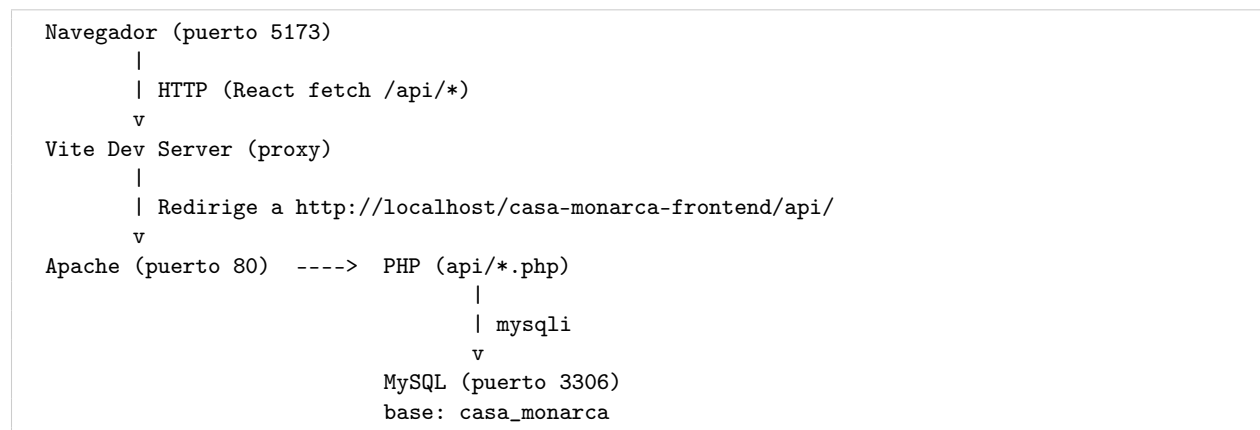


Figura 2.1: Diagrama de arquitectura del sistema

2.3. Componentes principales

Tabla 2.1: Componentes principales del sistema

Componente	Responsabilidad
src/App.jsx	Configuración de rutas con React Router. Define qué páginas requieren autenticación y qué roles pueden acceder.
ProtectedRoute.jsx	Guard de ruta. Valida sesión contra el backend antes de renderizar la página protegida.
pages/Login.jsx	Pantalla de inicio de sesión. Llama a /api/login.php.
pages/Dashboard.jsx	Menú principal. Muestra las opciones disponibles según el rol del usuario.
pages/AdminPanel.jsx	Gestión de usuarios y generación de llaves RSA para coordinadores.
pages/GestorDocumentos.jsx	Módulo central. Subida, consulta, aprobación, firma y verificación de documentos.
pages/Migrantes.jsx	Consulta y búsqueda del registro de migrantes.
pages/CrearMigrante.jsx	Formulario de registro de nuevo migrante.
pages/SolicitudesPush.jsx	Flujo de solicitudes delegadas entre usuarios y coordinadores.
api/*.php	Endpoints del backend. Cada archivo corresponde a una operación del sistema.
conexion.php	Configura la conexión a MySQL mediante <code>mysqli</code> .
database.sql	Schema completo de la base de datos con datos iniciales de prueba.

Capítulo 3

Stack Tecnológico

3.1. Lenguajes

Tabla 3.1: Lenguajes de programación utilizados

Lenguaje	Uso
JavaScript (ES2022+)	Lógica del frontend. Componentes React, manejo de estado, llamadas fetch.
JSX	Sintaxis de plantillas de React, compilada por Vite + Babel.
PHP 7.4+	Backend: endpoints de API, autenticación, firma digital (extensión OpenSSL).
SQL (MySQL)	Definición del esquema de base de datos y consultas con prepared statements.
CSS	Estilos globales y específicos de módulo.

3.2. Frameworks y herramientas principales

Tabla 3.2: Frameworks y herramientas del proyecto

Tecnología	Versión	Uso
React	19.2.4	Librería de UI para construir la SPA.
React Router DOM	7.14.1	Navegación entre páginas y guards de ruta.
Vite	8.0.4	Servidor de desarrollo y empaquetador de producción.
Bootstrap	5.3.8	Sistema de grilla y componentes UI (botones, tablas, modales).
XAMPP	Cualquier	Entorno local: Apache + MySQL + PHP integrados.

3.3. Bibliotecas y dependencias

Tabla 3.3: Dependencias del proyecto (package.json)

Paquete	Tipo	Uso
react	Producción	Núcleo de React.
react-dom	Producción	Renderizado en el DOM.
react-router-dom	Producción	Enrutamiento del SPA.
bootstrap	Producción	Estilos y componentes UI.
@vitejs/plugin-react	Desarrollo	Plugin de Vite para JSX y Fast Refresh.
eslint	Desarrollo	Análisis estático del código.
eslint-plugin-react-hooks	Desarrollo	Reglas de hooks de React.
@types/react	Desarrollo	Tipos TypeScript para autocompletado.

Capítulo 4

Estructura del Proyecto

4.1. Organización de carpetas

```
MA2006B/
+-- casa-monarca-frontend/      # Raiz del proyecto web
|   +-- api/                    # Backend: endpoints PHP (23 archivos)
|   +-- src/                    # Frontend React
|       +-- pages/              # Paginas de la aplicacion (12 archivos)
|       +-- components/        # Componentes reutilizables
|           | +-- ProtectedRoute.jsx
|           +-- App.jsx         # Configuracion de rutas
|           +-- main.jsx        # Punto de entrada React
|           +-- App.css         # Estilos globales
|           +-- index.css       # Variables CSS base
|   +-- public/                 # Assets estaticos
|   +-- uploads/               # Archivos subidos (generados en ejecucion)
|       +-- documentos/        # Documentos del gestor
|   +-- conexion.php           # Configuracion de conexion a MySQL
|   +-- index.html             # Punto de entrada HTML
|   +-- vite.config.js         # Configuracion de Vite y proxy
|   +-- package.json           # Dependencias de Node.js
+-- database.sql               # Esquema completo de la base de datos
+-- manual_usuario/            # Documentacion: manual de usuario
+-- documentacion/             # Documentacion: manual de desarrollo
+-- README.md                  # Guia rapida de instalacion
```

Figura 4.1: Estructura de carpetas del proyecto

4.2. Descripción de módulos

4.2.1. Módulo `api/` — Backend PHP

Cada archivo en `api/` corresponde a un endpoint independiente del sistema. Todos comparten el mismo patrón: leen JSON de `php://input`, validan datos, consultan la base de datos con prepared statements y devuelven JSON.

Tabla 4.1: Endpoints del backend PHP

Archivo	Método	Función
login.php	POST	Autenticar usuario.
validar_sesion.php	POST	Verificar que la sesión sigue activa.
usuarios.php	GET	Listar usuarios del sistema.
crear_usuario.php	POST	Crear nuevo usuario con rol asignado.
editar_usuario.php	POST	Editar datos de un usuario existente.
eliminar_usuario.php	POST	Desactivar un usuario.
coordinadores.php	GET	Listar usuarios con rol coordinador.
migrantes.php	GET	Listar migrantes registrados.
crear_migrante_directo.php	POST	Registrar un nuevo migrante.
documentos.php	GET	Listar documentos según rol/coordinador.
crear_documento.php	POST	Subir un documento e iniciarlo en etapa V.
avanzar_documento.php	POST	Aprobar o hacer inserción final de documento.
rechazar_documento.php	POST	Rechazar un documento del flujo.
eliminar_documento.php	POST	Eliminar un documento del sistema.
firmar_documento.php	POST	Firmar digitalmente con clave privada RSA.
verificar_firma_documento.php	POST	Verificar la firma digital de un documento.
generar_llaves_coordinador.php	POST	Generar par de llaves RSA para coordinador.
verificar_coordinador.php	POST	Verificar identidad del coordinador al iniciar sesión.
solicitudes_push.php	GET	Listar solicitudes push.
crear_solicitud_push.php	POST	Crear una solicitud push.
aprobar_solicitud_push.php	POST	Aprobar una solicitud push.
rechazar_solicitud_push.php	POST	Rechazar una solicitud push.

4.2.2. Módulo src/pages/ — Páginas React

Tabla 4.2: Páginas del frontend React

Archivo	Función
Login.jsx	Formulario de inicio de sesión.
Dashboard.jsx	Menú principal con accesos según rol.
AdminPanel.jsx	Lista de usuarios, generación de llaves RSA.
CrearUsuario.jsx	Formulario de alta de usuario.
EditarUsuario.jsx	Formulario de edición de usuario.
GestorDocumentos.jsx	Módulo completo del gestor de documentos.
Migrantes.jsx	Tabla de migrantes con búsqueda y filtros.
CrearMigrante.jsx	Formulario de registro de migrante.
SolicitudesPush.jsx	Módulo de solicitudes delegadas.
VerificacionCoordinador.jsx	Pantalla de verificación de identidad del coordinador.

Capítulo 5

Configuración del Entorno

5.1. Requisitos de versiones

Tabla 5.1: Requisitos de software por componente

Software	Versión mínima	Dónde obtenerlo
Node.js	v20.19 o v22.12+	https://nodejs.org
npm	Incluido con Node.js	—
XAMPP	Cualquier versión estable	https://www.apachefriends.org
Apache	2.4+ (incluido en XAMPP)	—
PHP	7.4+ (incluido en XAMPP)	—
MySQL/MariaDB	5.7+ (incluido en XAMPP)	—

5.2. Variables de entorno

El proyecto **no utiliza archivos** .env en la versión actual. La configuración de la conexión a la base de datos se realiza directamente en `conexion.php`:

Listing 5.1: Archivo `conexion.php` — variables de configuración

```
1 <?php
2 $host      = "localhost";
3 $usuario   = "CAMBIAR_USUARIO";      // usuario de MySQL
4 $password  = "CAMBIAR_CONTRASENA";  // contraseña de MySQL
5 $base_datos = "casa_monarca";
6
7 $conexion = new mysqli($host, $usuario, $password, $base_datos);
8 $conexion->set_charset("utf8");
9 ?>
```

Importante: nunca almacenar credenciales reales en el repositorio. Configurar este archivo de forma local antes de ejecutar o desplegar el sistema.

5.3. Archivos de configuración

Tabla 5.2: Archivos de configuración del proyecto

Archivo	Propósito
<code>conexion.php</code>	Credenciales de MySQL. Debe editarse por entorno.
<code>vite.config.js</code>	Proxy de desarrollo: redirige <code>/api/*</code> a Apache en <code>http://localhost</code> .
<code>package.json</code>	Dependencias y scripts npm del proyecto.
<code>eslint.config.js</code>	Reglas de análisis estático del código.
<code>database.sql</code>	Schema completo. Ejecutar una vez por entorno para crear la base de datos.

El proxy en `vite.config.js` funciona únicamente durante el desarrollo con `npm run dev`. En producción, el frontend y el backend deben estar en el mismo servidor Apache o configurar CORS manualmente.

Capítulo 6

Instalación y Ejecución

6.1. Pasos para levantar el sistema

1. **Instalar XAMPP** y verificar que Apache y MySQL puedan iniciarse correctamente.

2. **Clonar o copiar el proyecto** en el directorio raíz de XAMPP:

```
C:\xampp\htdocs\casa-monarca-frontend\
```

Esto es necesario para que Apache sirva los archivos PHP desde la ruta correcta.

3. **Iniciar Apache y MySQL** desde el panel de control de XAMPP. Ambos deben aparecer en verde.

4. **Crear la base de datos** abriendo <http://localhost/phpmyadmin>, ir a la pestaña **SQL**, pegar el contenido de `database.sql` y ejecutar.

5. **Configurar la conexión** editando `casa-monarca-frontend/conexion.php` con las credenciales de MySQL del entorno local.

6. **Instalar dependencias de Node.js** dentro de la carpeta `casa-monarca-frontend`:

```
npm ci
```

El comando `npm ci` utiliza `package-lock.json` para instalar versiones exactas y garantizar reproducibilidad.

7. **Iniciar el servidor de desarrollo:**

```
npm run dev
```

Vite levanta el frontend en <http://localhost:5173>.

8. **Verificar el sistema** accediendo a <http://localhost:5173> e iniciando sesión con las credenciales del usuario de prueba:

- Email: `admin@demo.com`
- Contraseña: `casa_monarca_pass`

6.2. Comandos principales

Tabla 6.1: Comandos npm disponibles

Comando	Descripción
npm ci	Instala dependencias exactas desde <code>package-lock.json</code> .
npm run dev	Inicia servidor de desarrollo en el puerto 5173 con proxy a Apache.
npm run build	Genera el bundle de producción en la carpeta <code>dist/</code> .
npm run preview	Sirve el build de producción localmente para validación.
npm run lint	Ejecuta ESLint sobre todo el código fuente.

Capítulo 7

Base de Datos

7.1. Descripción general

La base de datos se llama `casa_monarca` y está gestionada con MySQL/MariaDB. Se define completamente en el archivo `database.sql` ubicado en la raíz del proyecto. Contiene 5 tablas relacionales que cubren usuarios, migrantes, documentos, historial de cambios, firmas y solicitudes.

7.2. Tablas principales

7.2.1. Tabla `usuarios`

Almacena todos los usuarios del sistema con sus credenciales, rol, estado y llaves criptográficas para los coordinadores.

Listing 7.1: Definición de la tabla `usuarios`

```
1 CREATE TABLE IF NOT EXISTS usuarios (  
2     id                INT AUTO_INCREMENT PRIMARY KEY,  
3     nombre            VARCHAR(255) NOT NULL,  
4     email             VARCHAR(255) NOT NULL UNIQUE,  
5     password          VARCHAR(255) NOT NULL, -- bcrypt hash  
6     rol               VARCHAR(50) NOT NULL,  -- admin/coordinador/operador/  
7         consulta  
8     estado            VARCHAR(50) DEFAULT 'activo',  
9     fecha_vigencia    DATE NULL,  
10    certificado_codigo VARCHAR(100) NULL,  
11    clave_coordinador  VARCHAR(100) NULL,  
12    coordinador_id     INT NULL,             -- FK logica a usuarios  
13    clave_publica       TEXT NULL,           -- PEM RSA publica  
14    clave_privada_cifrada TEXT NULL,         -- PEM RSA cifrada con  
        contrasena  
);
```

7.2.2. Tabla `migrantes`

Almacena la información de cada migrante atendido por Casa Monarca.

Listing 7.2: Definición de la tabla `migrantes`

```

1 CREATE TABLE IF NOT EXISTS migrantes (
2     id                INT AUTO_INCREMENT PRIMARY KEY,
3     nombre            VARCHAR(255) NOT NULL,
4     apellido_paterno  VARCHAR(255) NULL,
5     apellido_materno  VARCHAR(255) NULL,
6     fecha_nacimiento  DATE          NULL,
7     edad              INT            NULL,
8     sexo              VARCHAR(50)    DEFAULT 'no_especificado',
9     nacionalidad      VARCHAR(100)   NULL,
10    telefono           VARCHAR(50)    NULL,
11    email              VARCHAR(255)   NULL,
12    pais_origen        VARCHAR(100)   NULL,
13    pais_destino       VARCHAR(100)   NULL,
14    estatus_migratorio VARCHAR(100)   NULL,
15    motivo_atencion   TEXT            NULL,
16    observaciones      TEXT            NULL,
17    creado_por         INT            NULL,    -- FK a usuarios.id
18    solicitud_id       INT            NULL,
19    creado_en          TIMESTAMP      DEFAULT CURRENT_TIMESTAMP
20 );

```

7.2.3. Tabla documentos

Registra cada documento del sistema con su etapa actual, archivo físico, hash de integridad y datos de firma.

Listing 7.3: Definición de la tabla documentos

```

1 CREATE TABLE IF NOT EXISTS documentos (
2     id                INT AUTO_INCREMENT PRIMARY KEY,
3     titulo            VARCHAR(255) NOT NULL,
4     descripcion       TEXT            NULL,
5     archivo_nombre    VARCHAR(255)   NULL,
6     archivo_ruta      VARCHAR(500)   NULL,
7     etapa            VARCHAR(10)     DEFAULT 'V', -- V/O/C/A
8     coordinador_id    INT            NULL,    -- FK a usuarios.id
9     creado_por        INT            NULL,    -- FK a usuarios.id
10    hash_archivo       VARCHAR(64)    NULL,    -- SHA-256 del archivo
11    estado_documento   VARCHAR(50)    DEFAULT 'activo',
12    firma_coordinador  TEXT            NULL,    -- firma RSA en Base64
13    folio_firma        VARCHAR(100)   NULL,
14    accion_firma       VARCHAR(255)   NULL,
15    firmado_por        INT            NULL,    -- FK a usuarios.id
16    fecha_firma        DATETIME       NULL
17 );

```

7.2.4. Tabla documentos_historial

Registra cada cambio de etapa de un documento, con el usuario responsable y un comentario.

7.2.5. Tabla documentos_firmas

Gestiona las firmas múltiples por documento. Permite que un coordinador solicite firmas adicionales de otros coordinadores antes de la inserción final.

Listing 7.4: Definición de documentos_firmas

```

1 CREATE TABLE IF NOT EXISTS documentos_firmas (
2     id                INT AUTO_INCREMENT PRIMARY KEY,
3     documento_id      INT                NOT NULL,
4     coordinador_id    INT                NOT NULL,
5     solicitado_por    INT                NULL,
6     estado            VARCHAR(50) DEFAULT 'pendiente', -- pendiente/firmado
7     hash_archivo      VARCHAR(64) NULL,
8     firma             TEXT                NULL,
9     folio_firma       VARCHAR(100) NULL,
10    fecha_firma       DATETIME           NULL,
11    UNIQUE KEY unique_doc_coord (documento_id, coordinador_id)
12 );

```

7.2.6. Tabla solicitudes_push

Gestiona solicitudes de acción delegadas entre usuarios y coordinadores (datos en formato JSON).

7.3. Relaciones entre tablas

Tabla 7.1: Relaciones entre tablas de la base de datos

Tabla origen	Tabla destino	Relación
usuarios.coordinador_id	usuarios.id	Operador → su Coordinador
documentos.creado_por	usuarios.id	Documento → creador
documentos.coordinador_id	usuarios.id	Documento → coordinador asignado
documentos.firmado_por	usuarios.id	Documento → quién firmó
documentos_historial.documento_id	documentos.id	Historial → documento
documentos_historial.usuario_id	usuarios.id	Historial → usuario
documentos_firmas.documento_id	documentos.id	Firma múltiple → documento
documentos_firmas.coordinador_id	usuarios.id	Firma múltiple → coordinador
migrantes.creado_por	usuarios.id	Migrante → usuario registrador
solicitudes_push.usuario_id	usuarios.id	Solicitud → usuario
solicitudes_push.coordinador_id	usuarios.id	Solicitud → coordinador

Capítulo 8

API / Interfaces

8.1. Convención general

Todos los endpoints siguen la misma convención:

- **URL base:** /api/<nombre_endpoint>.php
- **Content-Type:** application/json
- **Autenticación:** el usuario_id se envía en el cuerpo de la petición (sin JWT; la sesión se valida mediante validar_sesion.php).
- **Respuesta exitosa:** {"success": true, ...}
- **Respuesta de error:** {"success": false, "mensaje": "Descripción del error"}

8.2. Endpoints principales

8.2.1. POST /api/login.php

Autentica un usuario con email y contraseña.

Tabla 8.1: Parámetros de login.php

Parámetro	Tipo	Descripción
email	string	Correo electrónico del usuario.
password	string	Contraseña en texto plano (comparada contra bcrypt).

Respuesta exitosa:

```
{
  "success": true,
  "mensaje": "Login correcto",
  "usuario": {
    "id": 1, "nombre": "Administrador",
    "email": "admin@demo.com", "rol": "admin",
    "estado": "activo", "fecha_vigencia": "2027-12-31"
  }
}
```

8.2.2. POST /api/avanzar_documento.php

Avanza un documento de una etapa a la siguiente ($V \rightarrow O$ o $C \rightarrow A$).

Tabla 8.2: Parámetros de avanzar_documento.php

Parámetro	Tipo	Descripción
documento_id	int	ID del documento a avanzar.
usuario_id	int	ID del usuario que realiza la acción.
comentario	string	Comentario opcional para el historial.

8.2.3. POST /api/firmar_documento.php

Firma digitalmente un documento usando la clave privada RSA del coordinador.

Tabla 8.3: Parámetros de firmar_documento.php

Parámetro	Tipo	Descripción
documento_id	int	ID del documento a firmar.
usuario_id	int	ID del coordinador firmante.
password_firma	string	Contraseña para descifrar la clave privada RSA.
coordinadores_adicionales	array	IDs de coordinadores adicionales (opcional).

Respuesta exitosa:

```
{
  "success": true,
  "mensaje": "Documento firmado correctamente",
  "etapa": "C",
  "folio_firma": "FIR-20260430-120001-D0C3-USR2",
  "coordinadores_adicionales": []
}
```

8.2.4. POST /api/generar_llaves_coordinador.php

Genera un par de llaves RSA 2048 bits para un coordinador y almacena la clave privada cifrada con contraseña.

Tabla 8.4: Parámetros de generar_llaves_coordinador.php

Parámetro	Tipo	Descripción
solicitante_id	int	ID del usuario que solicita la generación (admin o el propio coordinador).
coordinador_id	int	ID del coordinador para quien se generan las llaves.
password_firma	string	Contraseña de cifrado de la clave privada (mínimo 6 caracteres).

Capítulo 9

Flujos y Lógica de Negocio

9.1. Flujo documental VOCA

El proceso central del sistema es el flujo de aprobación de documentos, conocido como **VOCA**:

```
[V - Ventanilla]
Documento creado/recibido
Acción: Operador o Admin aprueba
    |
    v
[O - Operador / Verificacion]
Documento en revision operativa
Acción: Coordinador firma con clave privada RSA
    |
    v
[C - Coordinador / Firma]
Documento firmado digitalmente
Acción: Admin realiza insercion final
    |
    v
[A - Insercion final]
Documento finalizado. Proceso completado.
```

Figura 9.1: Flujo documental VOCA

En cualquier etapa, un documento puede ser **rechazado**, lo que cambia su estado_documento a rechazado y bloquea cualquier acción posterior sobre él.

9.2. Reglas de negocio importantes

1. **Control por rol:** cada etapa del flujo solo puede ser ejecutada por el rol autorizado. Un coordinador no puede aprobar en Ventanilla (eso lo hace el operador o admin). Un operador no puede hacer inserción final.
2. **Control por coordinación:** los operadores y usuarios de consulta solo ven y actúan sobre documentos cuyo `coordinador_id` coincide con su propio `coordinador_id`.
3. **Integridad del archivo:** al firmar, el sistema calcula el hash SHA-256 del archivo físico y lo firma

con OpenSSL. La verificación posterior compara el hash actual del archivo con la firma almacenada, detectando cualquier modificación.

4. **Firmas múltiples:** el coordinador principal puede solicitar firmas adicionales de otros coordinadores. El documento solo puede avanzar a Inserción final cuando todas las firmas solicitadas estén en estado firmado.
5. **Vigencia del usuario:** al iniciar sesión, el sistema verifica que la `fecha_vigencia` del usuario no haya expirado. Si expiró, el acceso es denegado.
6. **Estado del usuario:** solo usuarios en estado `activo` pueden iniciar sesión.

9.3. Proceso de firma digital RSA

1. Al crear un coordinador, el admin (o el propio coordinador) ejecuta `generar_llaves_coordinador.php`. Se genera un par RSA 2048 bits con OpenSSL.
2. La clave pública se almacena en texto PEM en `usuarios.clave_publica`.
3. La clave privada se cifra con la contraseña elegida y se almacena en `usuarios.clave_privada_cifrada`.
4. Al firmar un documento, `firmar_documento.php`: calcula SHA-256 del archivo, descifra la clave privada con la contraseña, firma el hash con `openssl_sign` y almacena la firma en Base64 + folio único.
5. La verificación (`verificar_firma_documento.php`) recalcula el hash del archivo y lo verifica contra la firma almacenada usando la clave pública.

Capítulo 10

Ejemplos de Uso

Este capítulo presenta casos de uso representativos del sistema, con ejemplos concretos de llamadas al backend, respuestas esperadas y fragmentos de código del frontend. El objetivo es que un desarrollador que tome el proyecto por primera vez pueda comprender rápidamente cómo interactúan las capas del sistema.

10.1. Autenticar un usuario (login)

Caso típico: el usuario ingresa su correo y contraseña en la pantalla de login; el frontend envía las credenciales al backend y recibe los datos de sesión.

Llamada desde el frontend (JavaScript):

Listing 10.1: Llamada a login.php desde Login.jsx (elaboración propia).

```
1 const response = await fetch("/api/login.php", {
2   method: "POST",
3   headers: { "Content-Type": "application/json" },
4   body: JSON.stringify({ email, password })
5 });
6 const data = await response.json();
7
8 if (data.success) {
9   localStorage.setItem("usuario", JSON.stringify(data.usuario));
10  navigate("/dashboard");
11 } else {
12   setError(data.mensaje);
13 }
```

Respuesta exitosa del servidor:

Listing 10.2: Respuesta JSON de login.php para credenciales válidas (elaboración propia).

```
{
  "success": true,
  "mensaje": "Login correcto",
  "usuario": {
    "id": 1,
    "nombre": "Administrador",
    "email": "admin@demo.com",
    "rol": "admin",
    "estado": "activo",
```

```

    "fecha_vigencia": "2027-12-31",
    "certificado_codigo": "A3F9D2C18E4B"
  }
}

```

Respuesta cuando la contraseña es incorrecta:

Listing 10.3: Respuesta JSON de login.php para credenciales inválidas (elaboración propia).

```

{ "success": false, "mensaje": "Contraseña incorrecta." }

```

10.2. Subir y avanzar un documento

Caso típico: el usuario de consulta sube un archivo y el operador lo aprueba desde Ventanilla.

Paso 1 — Subir el documento (POST a crear_documento.php con FormData):

Listing 10.4: Subida de documento con FormData desde GestorDocumentos.jsx (elaboración propia).

```

1 const formData = new FormData();
2 formData.append("titulo", titulo);
3 formData.append("descripcion", descripcion);
4 formData.append("archivo", archivoSeleccionado);
5 formData.append("usuario_id", usuario.id);
6
7 const res = await fetch("/api/crear_documento.php", {
8   method: "POST",
9   body: formData          // sin Content-Type: el navegador lo fija solo
10 });
11 const data = await res.json();
12 // data.success === true -> documento en etapa "V"

```

Paso 2 — Aprobar desde Ventanilla (POST a avanzar_documento.php):

Listing 10.5: Aprobación de documento V→O desde GestorDocumentos.jsx (elaboración propia).

```

1 const res = await fetch("/api/avanzar_documento.php", {
2   method: "POST",
3   headers: { "Content-Type": "application/json" },
4   body: JSON.stringify({
5     documento_id: doc.id,
6     usuario_id: usuario.id,
7     comentario: "Revisado y aprobado"
8   })
9 });
10 const data = await res.json();
11 // data.etapa === "O" -> documento avanzó a Operador/Verificación

```

10.3. Firmar un documento digitalmente

Caso típico: el coordinador selecciona un documento en etapa O e ingresa su contraseña de firma para generar la firma RSA.

Precondición: el coordinador debe tener llaves RSA generadas (clave_publica y clave_privada_cifrada no nulos en la tabla usuarios).

Listing 10.6: Firma digital de documento desde GestorDocumentos.jsx (elaboración propia).

```
1 const res = await fetch("/api/firmar_documento.php", {
2   method: "POST",
3   headers: { "Content-Type": "application/json" },
4   body: JSON.stringify({
5     documento_id: doc.id,
6     usuario_id: usuario.id,
7     password_firma: passwordFirma,
8     coordinadores_adicionales: [] // sin firmas adicionales
9   })
10 });
11 const data = await res.json();
12
13 if (data.success) {
14   // data.folio_firma -> "FIR-20260612-143022-D0C5-USR2"
15   // data.etapa -> "C"
16   mostrarTicket(data.folio_firma);
17 }
```

Respuesta exitosa:

Listing 10.7: Respuesta de firmar_documento.php tras firma RSA exitosa (elaboración propia).

```
{
  "success": true,
  "mensaje": "Documento firmado correctamente",
  "etapa": "C",
  "folio_firma": "FIR-20260612-143022-D0C5-USR2",
  "coordinadores_adicionales": []
}
```

10.4. Generar llaves RSA para un coordinador

Caso típico: el administrador genera por primera vez el par de llaves RSA de un coordinador desde el Panel de Administración.

Listing 10.8: Generación de llaves RSA-2048 para un coordinador (elaboración propia).

```
1 const res = await fetch("/api/generar_llaves_coordinador.php", {
2   method: "POST",
3   headers: { "Content-Type": "application/json" },
4   body: JSON.stringify({
5     solicitante_id: usuarioActual.id, // admin o el propio coordinador
6     coordinador_id: coordinadorSeleccionado.id,
7     password_firma: nuevaPasswordFirma // mínimo 6 caracteres
8   })
9 });
10 const data = await res.json();
11 // data.success === true -> llaves almacenadas en usuarios.clave_publica
12 // y usuarios.clave_privada_cifrada
```

- Si solicitante_id es un **admin**, puede generar llaves para cualquier coordinador.
- Si solicitante_id es un **coordinador**, solo puede generar sus propias llaves (solicitante_id === coordinador_id).

- Si el coordinador ya tenía llaves, estas son **reemplazadas**. Los documentos firmados con las llaves anteriores siguen siendo verificables porque la firma y la clave pública anterior permanecen en la tabla `documentos_firmas`.

10.5. Verificar la integridad de un documento firmado

Caso típico: cualquier usuario quiere confirmar que un documento no fue alterado después de ser firmado.

Listing 10.9: Verificación de firma RSA/SHA-256 desde el frontend (elaboración propia).

```
1 const res = await fetch("/api/verificar_firma_documento.php", {
2   method: "POST",
3   headers: { "Content-Type": "application/json" },
4   body: JSON.stringify({ documento_id: doc.id })
5 });
6 const data = await res.json();
7
8 if (data.valida) {
9   mostrarMensaje("Firma válida. El documento no fue modificado.");
10 } else {
11   mostrarAlerta("Firma inválida o documento alterado.");
12 }
```

El backend recalcula el hash SHA-256 del archivo físico actual y lo compara contra la firma RSA almacenada usando la clave pública del coordinador firmante. Si el archivo fue modificado en cualquier byte posterior a la firma, la verificación falla.

Capítulo 11

Manejo de Errores

11.1. Estrategia de errores

El backend siempre devuelve una respuesta JSON estructurada, nunca HTML de error de PHP. Esto se garantiza con:

- `ob_start()` al inicio de cada endpoint para capturar cualquier salida accidental.
- `ob_clean()` antes de cada `echo json_encode()` para garantizar JSON puro.
- `ini_set("display_errors", 0)` para no exponer stack traces a los clientes.

El frontend interpreta el campo `success` de cada respuesta. Si es `false`, muestra el mensaje al usuario mediante alertas de Bootstrap.

11.2. Mensajes de error comunes

Tabla 11.1: Errores comunes del sistema y causa raíz

Mensaje	Causa y solución
<i>Datos incompletos</i>	Faltan campos requeridos en la petición. Revisar el cuerpo del request.
<i>Usuario no encontrado</i>	Email o ID no existe en la base de datos.
<i>Contraseña incorrecta</i>	Hash bcrypt no coincide. Verificar datos de acceso.
<i>Usuario inactivo o revocado</i>	Campo estado distinto de activo. Activar desde AdminPanel.
<i>La vigencia ha expirado</i>	<code>fecha_vigencia</code> anterior a la fecha actual. Actualizar desde AdminPanel.
<i>Contraseña de firma incorrecta</i>	La contraseña no descifra la clave privada RSA.
<i>El coordinador no tiene llaves</i>	No se han generado llaves RSA. Usar AdminPanel para generarlas.
<i>No se puede avanzar un documento rechazado</i>	El documento tiene <code>estado_documento = rechazado</code> . No es recuperable.
<i>Hay firmas pendientes</i>	Existen coordinadores adicionales que aún no han firmado.
<i>Respuesta inválida del servidor</i>	Apache o PHP no está corriendo. Verificar XAMPP.

11.3. Registro (logs)

Los errores de PHP se registran en el log de Apache de XAMPP, ubicado normalmente en:

```
C:\xampp\apache\logs\error.log
```

El frontend registra errores de red y de parsing en la consola del navegador (herramientas de desarrollador).

Capítulo 12

Pruebas

12.1. Tipos de pruebas

En la versión 1.0 del sistema se realizaron exclusivamente **pruebas manuales funcionales** mediante la interfaz web. No se implementaron pruebas automatizadas unitarias ni de integración.

12.2. Casos de prueba manuales realizados

Tabla 12.1: Casos de prueba funcionales manuales

Caso	Resultado	Notas
Login con credenciales correctas	Aprobado	Redirige al dashboard.
Login con contraseña incorrecta	Aprobado	Muestra mensaje de error.
Login con usuario inactivo	Aprobado	Bloquea acceso.
Crear usuario con email duplicado	Aprobado	Retorna error.
Subir documento y verificar etapa V	Aprobado	Aparece en la tabla.
Aprobar documento como operador (V→O)	Aprobado	Etapa cambia correctamente.
Firmar documento como coordinador (O→C)	Aprobado	Folio generado.
Intentar firmar con contraseña incorrecta	Aprobado	Retorna error.
Inserción final como admin (C→A)	Aprobado	Documento finalizado.
Verificar firma digital	Aprobado	Firma válida confirmada.
Rechazar documento	Aprobado	Bloquea avance posterior.
Generar llaves RSA para coordinador	Aprobado	Llaves almacenadas.

12.3. Cómo ejecutar ESLint

```
cd casa-monarca-frontend
npm run lint
```

Estado conocido: ESLint reporta actualmente 20 errores y 2 advertencias relacionados principalmente con variables no utilizadas y dependencias de hooks. Esta deuda técnica no impide compilar ni ejecutar

el sistema, pero debe resolverse antes de implementar ESLint como validación obligatoria en el flujo de CI/CD.

Capítulo 13

Despliegue

13.1. Entornos

Tabla 13.1: Entornos del sistema

Entorno	Descripción
Desarrollo local	XAMPP + Vite dev server. Usar <code>npm run dev</code> . Proxy automático a Apache.
Producción	Servidor Apache con PHP. El frontend debe compilarse con <code>npm run build</code> y los archivos de <code>dist/</code> servirse desde Apache.
HostGator	Hosting compartido utilizado para el despliegue en producción del proyecto. Limitaciones indicadas en la sección siguiente.

13.2. Pasos para despliegue en producción

1. Generar el bundle de producción del frontend:

```
npm run build
```

Esto genera los archivos estáticos en la carpeta `dist/`.

2. Subir el contenido de `dist/` y la carpeta `api/` al servidor Apache de producción.
3. Configurar el archivo `conexion.php` con las credenciales de la base de datos de producción.
4. Ejecutar `database.sql` en el servidor MySQL de producción para crear el schema.
5. Verificar que la extensión `openssl` esté habilitada en PHP del servidor de producción (requerida para firma RSA).
6. Configurar los headers CORS en el servidor si el frontend y el backend están en dominios distintos.

13.3. Consideraciones y limitaciones conocidas (HostGator)

- **OpenSSL:** verificar que la extensión OpenSSL de PHP esté disponible y habilitada en el plan de hosting.

- **Ruta de archivos:** la ruta de `uploads/documentos/` debe existir y tener permisos de escritura para el usuario de Apache.
- **Sin acceso SSH en planes básicos:** las migraciones de base de datos deben realizarse desde phpMyAdmin del panel de control.
- **Versión de PHP:** confirmar que el servidor use PHP 7.4 o superior.
- **Sin variables de entorno del sistema:** las credenciales deben configurarse directamente en `conexion.php` (protegido del repositorio vía `.gitignore`).

Capítulo 14

Seguridad

14.1. Autenticación

- Las contraseñas se almacenan como hashes **bcrypt** con `password_hash()` de PHP.
- La verificación usa `password_verify()`, nunca comparación directa.
- Cada ruta protegida del frontend valida la sesión contra el backend mediante `validar_sesion.php` antes de renderizar el contenido.
- Los datos del usuario activo se almacenan en `localStorage` del navegador.

14.2. Firma digital

- Las llaves RSA son de **2048 bits** generadas con OpenSSL.
- La clave privada nunca viaja en texto claro: se almacena cifrada con la contraseña del coordinador.
- El algoritmo de firma es **SHA-256 con RSA** (`OPENSSL_ALGO_SHA256`).
- El hash SHA-256 del archivo se recalcula en cada verificación para detectar modificaciones posteriores a la firma.

14.3. Protección de datos sensibles

- El archivo `conexion.php` está en `.gitignore` para evitar que las credenciales reales sean incluidas en el repositorio.
- Las claves privadas cifradas se almacenan en la base de datos; solo se pueden usar con la contraseña correcta.
- Los endpoints PHP usan **prepared statements** con `mysqli` para prevenir inyección SQL.
- Los headers CORS están configurados con `Access-Control-Allow-Origin: *` en desarrollo; deben restringirse a dominios específicos en producción.

14.4. Buenas prácticas implementadas

- `ob_start()` / `ob_clean()` para garantizar respuestas JSON puras.
- `display_errors = 0` en producción para no exponer información del servidor.
- Validación de rol en cada endpoint del backend (no solo en el frontend).
- Control de vigencia y estado del usuario en cada autenticación.

Capítulo 15

Guía de Contribución

15.1. Convenciones de código

15.1.1. Frontend (JavaScript/React)

- Nombrar archivos de componentes con **PascalCase**: `GestorDocumentos.jsx`.
- Nombrar variables y funciones con **camelCase**: `handleSubmit`, `documentoId`.
- Usar **functional components** con hooks; no usar class components.
- Mantener la lógica de fetch dentro del componente que la necesita o en funciones auxiliares del mismo archivo.
- Cada página nueva debe añadirse al enrutador en `App.jsx` con un `ProtectedRoute` que especifique los roles permitidos.

15.1.2. Backend (PHP)

- Un archivo PHP por endpoint; el nombre debe describir la acción: `crear_usuario.php`.
- Usar siempre **prepared statements** con `mysqli`; nunca concatenar variables de usuario en SQL.
- Iniciar siempre con `ob_start()` y usar `ob_clean()` antes de cada respuesta.
- Toda respuesta debe ser JSON con el campo `success` (bool) y `mensaje` (string).

15.2. Flujo de trabajo con Git

1. Crear una rama descriptiva desde main:

```
git checkout -b feature/nombre-de-la-funcionalidad
```

2. Hacer commits con mensajes claros y en español:

```
git commit -m "Agrega endpoint para rechazar solicitudes push"
```

3. Antes de hacer push, revisar que el código no rompa el build:

```
npm run build
```

4. Abrir un Pull Request hacia `main` para revisión del equipo.
5. Nunca hacer push directo a `main` sin revisión.

15.3. Cómo agregar funcionalidades

Para agregar un nuevo módulo al sistema seguir estos pasos:

1. **Backend:** crear el archivo `api/nombre_accion.php` con la lógica del endpoint.
2. **Base de datos:** si se requiere nueva tabla, agregarla a `database.sql` con `CREATE TABLE IF NOT EXISTS`.
3. **Frontend:** crear la página en `src/pages/NombrePagina.jsx`.
4. **Ruta:** registrar la nueva ruta en `App.jsx` con `ProtectedRoute` y los roles correspondientes.
5. **Dashboard:** agregar el acceso visible en `Dashboard.jsx` para los roles autorizados.

Capítulo 16

Problemas Conocidos

16.1. Limitaciones de la versión 1.0

Tabla 16.1: Limitaciones conocidas del sistema

Limitación	Descripción
Sin autenticación basada en tokens (JWT/session)	La sesión se almacena en localStorage. En un entorno de mayor seguridad se recomienda migrar a tokens HTTP-only.
CORS abierto en desarrollo	Access-Control-Allow-Origin: * debe restringirse en producción.
Sin notificaciones por correo	El sistema no envía emails al crear usuarios, asignar documentos ni al firmar.
ESLint con deuda técnica	20 errores y 2 advertencias pendientes de resolver.
Ruta de OpenSSL fija en Windows	generar_llaves_coordinador.php tiene una ruta de configuración openssl.cnf para MAMP (Windows). En Linux/HostGator puede omitirse o ajustarse.
Sin paginación en tablas grandes	Las tablas de documentos, migrantes y usuarios cargan todos los registros sin paginación.

Capítulo 17

FAQ Técnica

¿Por qué el frontend no puede conectarse a la API?

Verificar que Apache esté corriendo en XAMPP (puerto 80) y que el proyecto esté ubicado en `htdocs/casa-monarca-frontend`. Abrir directamente `http://localhost/casa-monarca-frontend/api/login.php` para confirmar que PHP responde.

¿Por qué aparece Respuesta inválida del servidor al hacer login?

PHP está devolviendo HTML de error en lugar de JSON. Revisar `conexion.php`: las credenciales de MySQL deben coincidir con la instalación local. Abrir `http://localhost/casa-monarca-frontend/conexion.php` para ver el mensaje de error real.

¿Por qué falla la generación de llaves RSA?

En entornos Windows con MAMP, la ruta del archivo `openssl.cnf` en `generar_llaves_coordinador.php` debe apuntar a la instalación correcta. En servidores Linux o HostGator, puede eliminarse la clave `$config` del array `$config` ya que OpenSSL la detecta automáticamente.

¿Por qué el operador no puede ver los documentos?

El operador solo ve documentos cuyo `coordinador_id` coincide con su propio `coordinador_id`. Verificar que el usuario operador tenga un coordinador asignado en AdminPanel.

¿Cómo resetear la contraseña de firma de un coordinador?

Generar nuevas llaves RSA para el coordinador desde AdminPanel usando una contraseña nueva. Las llaves anteriores quedarán reemplazadas y los documentos firmados con las llaves antiguas seguirán siendo verificables mientras la firma esté almacenada en la base de datos.

¿Por qué el puerto 3306 da error al iniciar XAMPP?

Otro servicio de MySQL está corriendo. En Windows: Administrador de tareas → pestaña Servicios → buscar "MySQL" → Detener. Luego reiniciar MySQL desde el panel de XAMPP.

¿Cómo generar el build de producción?

```
cd casa-monarca-frontend
npm run build
```

Los archivos generados en `dist/` son los que deben subirse al servidor de producción junto con la carpeta `api/` y el archivo `conexion.php` configurado.

Capítulo 18

Referencias

- [1] Documentación oficial de React. Disponible en: <https://react.dev>
- [2] Documentación oficial de React Router v7. Disponible en: <https://reactrouter.com>
- [3] Documentación oficial de Vite. Disponible en: <https://vite.dev>
- [4] Bootstrap 5 — Componentes y utilidades. Disponible en: <https://getbootstrap.com/docs/5.3>
- [5] PHP Manual — openssl_sign y funciones de criptografía. Disponible en: <https://www.php.net/manual/es/book.openssl.php>
- [6] PHP Manual — mysqli y prepared statements. Disponible en: <https://www.php.net/manual/es/class.mysqli.php>
- [7] XAMPP — Entorno de desarrollo local. Disponible en: <https://www.apachefriends.org>
- [8] Node.js — Descargas oficiales. Disponible en: <https://nodejs.org>